

CMU-CS 462: Software Measurement and Analysis

Spring 2025-2026

Software Cost Metrics

Nguyen Duc Man
E: mannd@duytan.edu.vn
T: 0904235945

12/01/2026

1

1

Contents

1. Software cost model
2. COCOMO and COCOMO II
3. Constraint model
4. Software lifecycle management (SLIM)
5. Cost models: advantages and drawbacks

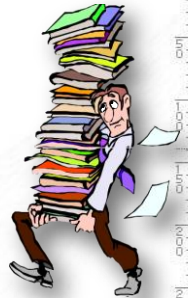
6. Software cost

Cost model

Effort vs. cost drive (size)

Constraint model

Effort & cost vs. time



2

Discussion – Software Cost Metrics

1. What are the key differences between the COCOMO and SLIM cost estimation models?
2. How does the COCOMO II model differ from the original COCOMO model?
3. What are the advantages and disadvantages of using Expert Judgment for software cost estimation?
4. What role do cost drivers play in software cost estimation, and how do they affect the accuracy of the model?
5. In the context of software cost estimation, what is Parkinson's Law, and how does it impact the estimation process?
6. What are the key challenges in size estimation for software projects, and how do they affect cost estimation?

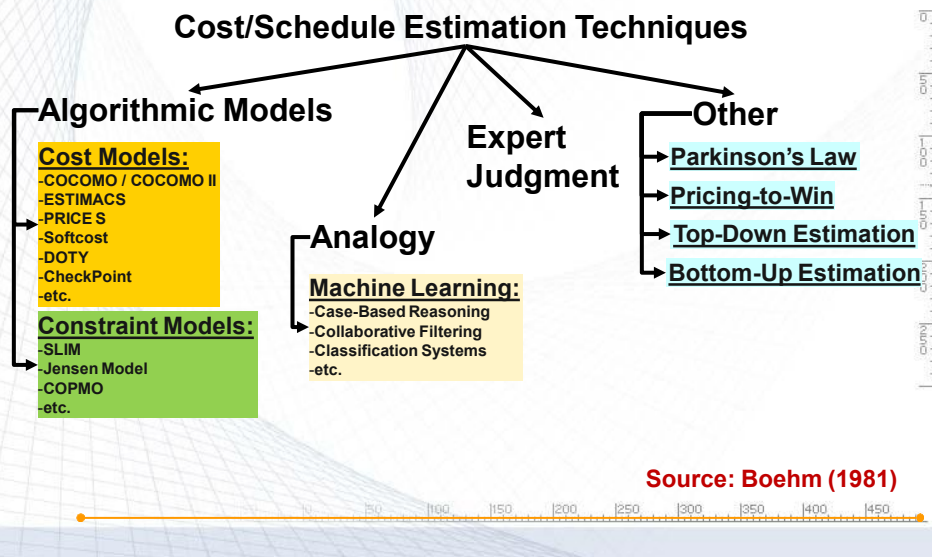
3

Software Cost Estimation

- Software cost estimation is the process of predicting the amount of **effort** required to build a software system and **time** to develop it.
- Models provide mathematical algorithms to compute cost as a function of a number of variables such as size (using lines of code, function points, etc.) and/or complexity (using cyclomatic complexity, etc.).
- Models used to estimate cost can be categorized as:
 - **Cost model**; e.g., COCOMO
 - **Constraint model**; e.g., SLIM
- Most of the models are available as automated tools.

4

Cost Estimation: 7 Techniques



5

Cost/Schedule Estimation Techniques /1

Bottom-Up Estimation:

- Start at the lowest system level. The cost/schedule of each component is estimated. All these costs are added to produce a final cost estimate.
- Advantage:
 - Can be accurate, if the system has been designed in detail
- Disadvantage:
 - May underestimate costs of system level activities such as integration and documentation

6

Cost/Schedule Estimation Techniques /2

Top-Down Estimation:

- A cost/schedule estimate is established by considering the overall functionality of the product and how that functionality is provided by interacting sub-functions. Cost estimates are made on the basis of the logical function rather than the components implementing that function.
- Advantage: Takes into account costs such as integration, configuration management and documentation
- Disadvantage: Can underestimate the cost of solving difficult low-level technical problems

7

Cost/Schedule Estimation Techniques /3

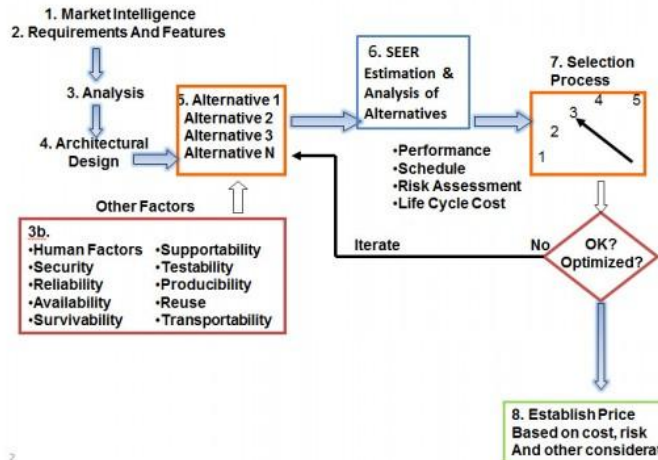
Pricing-to-Win

- Software cost/schedule is estimated to be whatever the customer has available to spend on the project. The estimated effort depends on the customer's budget and not on the software functionality.
- Advantage: Good chances to win the contract!
- Disadvantages: The probability that costs accurately reflect the work required and the probability that the customer gets the system he or she wants are low.

8

Cost/Schedule Estimation Techniques /3

Price-To-Win Flow & Responsibilities



9

Cost/Schedule Estimation Techniques /4

Parkinson's Law

- Parkinson's Law states that work expands to fill the time available. This means that cost is determined by available resources rather than objective assessment.
 - Example: If the software has to be delivered in 12 months and 5 people are available, the effort required is estimated to be 60 person-months.
- Advantage: No overspending
- Disadvantage: System is usually unfinished / or effort is wasted

10

Cost/Schedule Estimation Techniques /5

Expert Judgement

- One or more experts in both software development and the application domain use their experience to predict software costs. Process iterates until some consensus is reached (e.g., using Delphi Method).
- Advantage: Relatively cheap estimation method. Can be accurate if experts have substantial experience with similar systems.
- Disadvantage: Very inaccurate if there are no experts! Difficult to trace back, in case the estimate was not accurate.

11

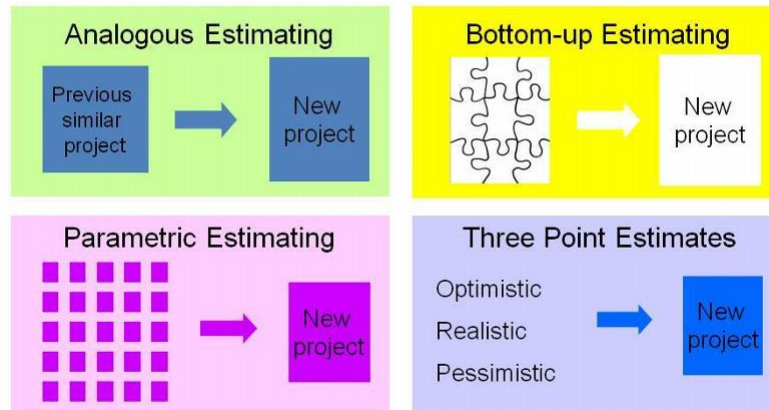
Cost/Schedule Estimation Techniques /6

Estimation by Analogy

- This technique is applicable when other projects in the same application domain have been done in the past. The cost of a new project is computed by comparing the project to a similar completed project in the same application domain
- Advantages: Accurate if similar projects are available
- Disadvantages: Impossible if no comparable project available. Needs systematically maintained project database (→ expensive). It's not always clear how to define a good similarity function.

12

Estimating - Methods



13

Algorithmic Models

- Uses historical cost information and relates it to some software project measure(s).
- Two main categories (Kitchenham, 1991):
 - **Empirical Factor Models (Cost Models):** Provide estimates of project cost (effort) and schedule based on size; derived from empirical data.
 - **Constraint Models:** Assume a more complex relationship between cost (effort), schedule and size, i.e., assume that productivity is not only a function of effort (input) and size (output) but that it is "constrained" by the (anticipated) schedule and/or complexity.

17

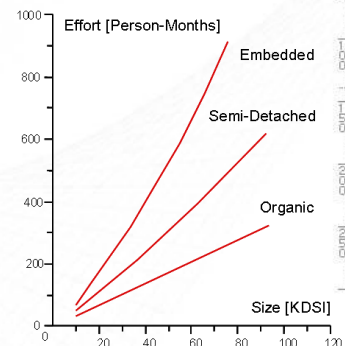
Cost Models /1

- Cost models provide direct estimates of *effort*.
- Cost models typically are based on:
 - a primary cost factor such as *size*
 - a number of secondary adjustment factors or *cost drivers*
- Cost drivers are characteristics of the project, process, products, or resources that influence effort.
- Cost drivers are used to adjust the preliminary estimate provided by the primary cost factor.

18

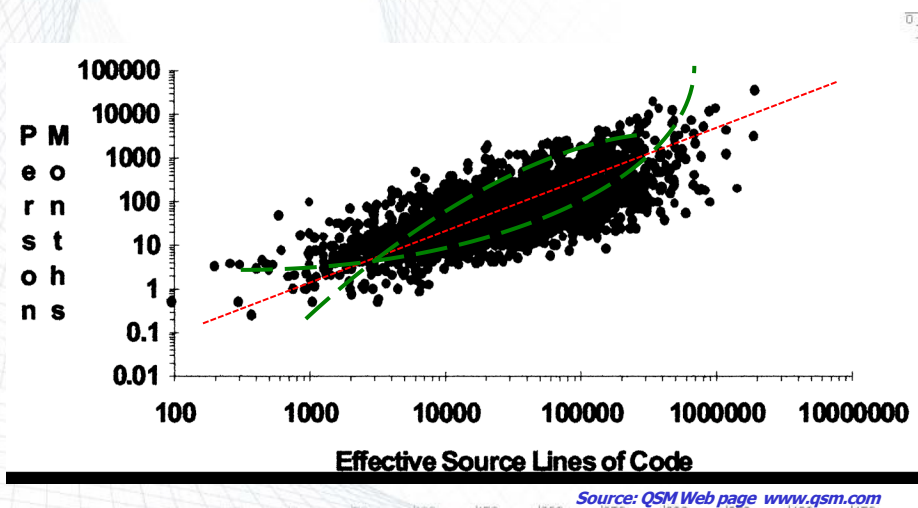
Cost Models /2

- *Effort* is plotted against the *primary cost factor* (usually LOC or FP) for a series of projects.
- The line of best fit is calculated for the data points. If the primary cost factor were a perfect predictor of effort, then every point on the graph would lie on the line of best fit.



19

Example



20

Cost Models /3

- The regression-based cost models take the form:

$$E = a + b \times S^c$$

where

- E is the effort in person months (PM)
- a , b , and c are empirically derived constants
- S is the primary cost factor (typically, either LOC or FP)

21

Cost Models /4

- The following are some examples of cost models using LOC as a primary cost factor (Pressman, 1997):

$$E = 5.2 \times (KLOC)^{0.91}$$

Walston-Felix Model

$$E = 5.5 + 0.73 \times (KLOC)^{1.16}$$

Bailey-Basili Model

$$E = 2.4 \times (KLOC)^{1.05}$$

COCOMO Basic Model

$$E = 5.288 \times (KLOC)^{1.047}$$

Doty Model for KLOC > 9

22

Cost Models /5

- The following are some examples of cost models using FP as a primary cost factor (Pressman, 1997):

$$E = -13.39 + 0.0545 FP$$

Albrecht and Gaffney Model

$$E = 60.62 \times 7.728 \times 10^{-8} FP^3$$

Kremerer Model

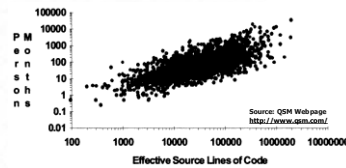
$$E = 585.7 + 15.12 FP$$

Matson, Barnett,
& Mellichamp Model

23

Constraint Models

- Constraint models demonstrate the relationship over time between two or more parameters of *effort*, *duration*, or *staffing level* (Fenton, 1997).
- Putnam's SLIM model is an example of constraint model: it adds time dimension to effort vs. size graph.



24

COCOMO

Prof. Barry Boehm

Constructive Cost Model

25

COCOMO /1

- **COCOMO (Constructive Cost Model)** (Boehm, 1981)
- COCOMO is a model based on inputs relating to the size of the system and a number of cost drivers that affect productivity.
- Updated version, called **COCOMO II**, accounts for recent changes in software engineering technology, including object-oriented software, software created via spiral or evolutionary development models, software reuse and building new systems using off-the-shelf software components.

26

COCOMO /2

- The original COCOMO is a collection of three models:
 - **Basic model** that is applied early in the project
 - **Intermediate model** that is applied after requirements acquisition
 - **Advanced model** that is applied after design is complete
- All three models take the form:

$$E = aS^b \times EAF \qquad T_{dev} = cE^d$$

where

- E is effort in person months
- T_{dev} is the development time
- S is size measured in KLOC
- EAF is an effort adjustment factor (1 in the Basic model)
- Factors a , b , c and d depend on the **development mode**.

27

COCOMO: Basic

- The Basic COCOMO model computes effort (person-month) as a function of program size. The Basic COCOMO equation is:

$$E_{nom} = a(KLOC)^b$$

$$EAF = 1$$

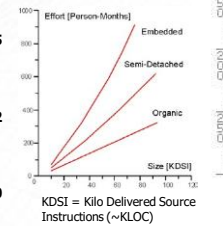
- The factors a and b for the Basic model are shown in the table.
- T_{dev} is not defined

Mode	a	b
Organic	2.4	1.05
Semi-detached	3.0	1.12
Embedded	3.6	1.20

Organic:
PM = 2.4 (KDSI) ^{1.05}

Semi-detached:
PM = 3.0 (KDSI) ^{1.12}

Embedded:
PM = 3.6 (KDSI) ^{1.20}



28

Basic COCOMO: Examples

- Organic mode project:** 32 KLOC
 - PM = 2.4 (32) ^{1.05} = 91 person-months
 - Development time: don't know
- Embedded mode project:** 128 KLOC
 - PM = 3.6 (128) ^{1.2} = 1216 person-months
 - Development time: don't know

29

COCOMO: Intermediate

/1

- The Intermediate COCOMO model computes effort as a function of program size and a set of cost drivers. The intermediate COCOMO equation is:

$$E = a(KLOC)^b \times EAF$$

$$T_{dev} = cE_{nom}^d$$

- The factors a , b , c and d for the intermediate model are shown in the table.

Mode	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

Mode	c	d
Organic	2.5	0.38
Semi-detached	2.5	0.35
Embedded	2.5	0.32

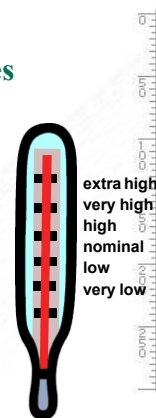
The effort adjustment factor (EAF) is calculated using **15 cost drivers**.

30

COCOMO: Intermediate

/2

- The effort adjustment factor (EAF) is calculated using 15 cost drivers. ← **project related features**
- The cost drivers are grouped into 4 categories: **product**, **platform**, **personnel**, and **project**.
- Each cost driver is rated on a 6 point ordinal scale ranging from *very low* to *extra high* importance. Based on the rating, the **effort multiplier (EM)** is determined (Boehm, 1981). The product of all effort multipliers is the EAF .



$$E = E_{nom} \times EAF$$

$$EAF = \prod_{i=1}^{15} EM_i$$

31

COCOMO Intermediate /3

15 Cost Drivers

- **Product Factors**
 - Reliability (RELY)
 - Data (DATA)
 - Complexity (CPLX)
- **Platform Factors**
 - Time constraint (TIME)
 - Storage constraint (STOR)
 - Platform volatility (PVOL)
- **Personnel factors**
 - Analyst capability (ACAP)
 - Program capability (PCAP)
 - Applications experience (APEX)
 - Platform experience (PLEX)
 - Language and tool experience (LTEX)
 - Personnel continuity (PCON)
- **Project Factors**
 - Software tools (TOOL)
 - Multisite development (SITE)
 - Required schedule (SCED)

32

Cost Driver Rating /1

1. Product

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
RELY	Required software reliability	0.75	0.88	1.00	1.15	1.40	-
DATA	Database size	-	0.94	1.00	1.08	1.16	-
CPLX	Product complexity	0.70	0.85	1.00	1.15	1.30	1.65

33

Cost Driver Rating /2

2. Platform

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
TIME	Execution time constraint	-	-	1.00	1.11	1.30	1.66
STOR	Main storage constraint	-	-	1.00	1.06	1.21	1.56
VIRT	Virtual machine volatility	-	0.87	1.00	1.15	1.30	-
TURN	Computer turnaround time	-	0.87	1.00	1.07	1.15	-

34

Cost Driver Rating /3

3. Personnel

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
ACAP	Analyst capability	1.46	1.19	1.00	0.86	0.71	-
AEXP	Applications experience	1.29	1.13	1.00	0.91	0.82	-
PCAP	Programmer capability	1.42	1.17	1.00	0.86	0.70	-
VEXP	Virtual machine experience	1.21	1.10	1.00	0.90	-	-
LEXP	Language experience	1.14	1.07	1.00	0.95	-	-

35

Cost Driver Rating /4

4. Project

	Description	Very Low	Low	Nominal	High	Very High	Extra High
MODP	Modern programming practices	1.24	1.10	1.00	0.91	0.82	-
TOOL	Software Tools	1.24	1.10	1.00	0.91	0.83	-
SCED	Development Schedule	1.23	1.08	1.00	1.04	1.10	-

36

Cost Driver Rating: Example

- **Example: Required Software Reliability (RELY)**
 - Measures the extent to which the software must perform its intended function over a period of time.
 - Ask: What is the *effect of a software failure*?



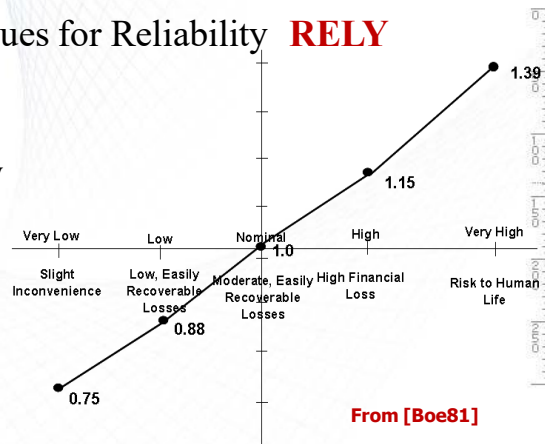
	Very Low	Low	Nominal	High	Very High	Extra High
RELY	slight inconvenience	low, easily recoverable losses	moderate, easily recoverable losses	high financial loss	risk to human life	
	0.75	0.88	1.00	1.15	1.39	

37

Cost Driver Rating: Example

■ Effort Multiplier values for Reliability **RELY**

- E.g., a highly reliable system costs 39% more than a nominally reliable system ($1.39/1.0 = 1.39$)
- A highly reliable system costs 85% more than a very low reliability system ($1.39/0.75 = 1.85$)

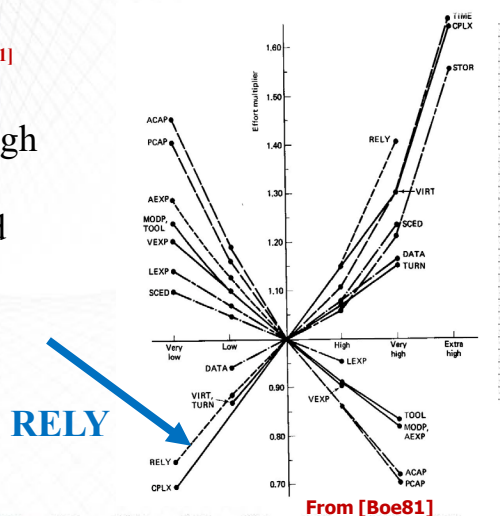


38

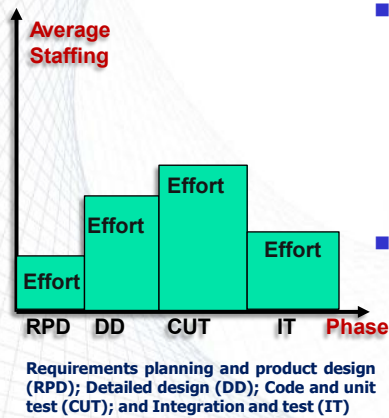
Cost Driver Rating /5

■ **Cost Driver Impact** [Boe81]

COCOMO provides a framework to identify high leverage productivity improvement factors and estimate their payoffs



COCOMO: Advanced /1



- Advanced COCOMO model computes effort as a function of program size and a set of cost drivers *weighted according to each phase of software lifecycle*.
- Advanced model applies the intermediate model at the component level, and then a phase-based approach is used to consolidate the estimate.

40

COCOMO: Advanced /2

- The 4 phases used in the detailed COCOMO model are: requirements planning and product design (RPD), detailed design (DD), code and unit test (CUT), and integration and test (IT).
- Each cost driver is broken down by phase as in the table below.
- Estimates made for each module are combined into subsystems and eventually an overall project estimate.

Cost Driver	Rating	RPD	DD	CUT	IT	ACAP Intermediate
ACAP (Analyst capability)	Very Low	1.80	1.35	1.35	1.50	1.46 – Very Low
	Low	0.85	0.85	0.85	1.20	1.19 – Low
	Nominal	1.00	1.00	1.00	1.00	1.00 – Nominal
	High	0.75	0.90	0.90	0.85	0.86 – High
	Very High	0.55	0.75	0.75	0.70	0.71 – Very High

41

COCOMO II

Constructive Cost Model II

<http://softwarecost.org/tools/COCOMO/>

<https://strs.grc.nasa.gov/repository/forms/cocomo-calculation/>

42

COCOMO II

- COCOMO II includes three-stage series of models:
 - The earliest phases will generally involve prototyping, using the *Application Composition* model capabilities. (← replaces Basic model in COCOMO)
 - COCOMO II provides an early estimation model called the *Early Design model*. (← replaces Intermediate model in COCOMO)
 - Once the project is ready to develop and sustain a fielded system, it should have a life-cycle architecture, which provides more accurate information on cost driver inputs, and enables more accurate cost estimates. To support this stage, COCOMO II provides the *Post- Architecture model*. (← replaces Advanced model in COCOMO)

43

1. Application Composition Model

- The Application Composition model is used in early development stages and prototyping to resolve potential high-risk issues such as user interfaces, software/system interaction, performance, or technology maturity.
- Object points** are used for sizing rather than the traditional LOC metric.

$$E = OP / PROD$$

where

- OP is the object point
- $PROD$ is the productivity rate

Developers' experience and capability	Very Low	Low	Nominal	High	Very High
PROD	4	7	13	25	50

44

2. Early Design Model

/1

- The Early Design model is used to evaluate alternative software/system architectures and concepts of operation. Unadjusted function point count (UFC) is used for sizing. This value is converted to KLOC.
- The Early Design model's equation is:

$$E = 2.45 \times KLOC \times EAF$$
- The effort adjustment factor (EAF) is calculated as in the original COCOMO model using 7 cost drivers

45

2. Early Design Model

/2

- The effort adjustment factor (EAF) for the Early Design model (obtained by combining the 17 Post-Architecture cost drivers)

	<i>Cost Driver</i>	<i>Description</i>	<i>Combined Post-Architecture Cost Drivers</i>
1	RCPX	Product reliability and complexity	RELY, DATA, CPLX, DOCU
2	RUSE	Required reuse	RUSE
3	PDIF	Platform difficulty	TIME, STOR, PVOL
4	PERS	Personnel capability	ACAP, PCAP, PCON
5	PREX	Personnel experience	AEXP, PEXP, LTEX
6	FCIL	Facilities	TOOL, SITE
7	SCED	Schedule	SCED

46

3. Post-Architecture Model

- The Post-Architecture model is used during the actual development and maintenance of a product.
- Function point (FP) or LOC can be used for sizing, with modifiers for reuse and software breakage.
- The Post-Architecture model includes a set of 17 cost drivers (*EAF*) and a set of 5 scale factors (*SF*) determining the projects scaling component.
- The Post-Architecture model equation is:

$$E = 2.45 \times (KLOC)^b \times EAF$$

$$b = 0.91 + 0.01 \sum_{j=1}^5 SF_j \quad 0.91 \leq b \leq 1.23$$

47

Scale Factors /1

Scale Factor <i>SF</i>		Very Low	Low	Nominal	High	Very High	Extra High
Precedentedness Degree to which system is new and past experience applies	PREC	6.20	4.96	3.72	2.48	1.24	0.00
Development/Flexibility Need to conform to specified requirements	FLEX	5.07	4.05	3.04	2.03	1.01	0.00
Architecture/Risk Resolution Degree of design thoroughness and risk elimination	RESL	7.07	5.65	4.24	2.83	1.41	0.00
Team Cohesion Need to synchronize stakeholders and minimize conflict	TEAM	5.48	4.38	3.29	2.19	1.10	0.00
Process Maturity SEI CMM process maturity rating	PMAT	7.80	6.24	4.68	3.12	1.56	0.00

48

Scale Factors /2

$$E = 2.45 \times (KLOC)^b \times EAF$$

$$b = 0.91 + 0.01 \sum_{i=1}^5 SF_i$$

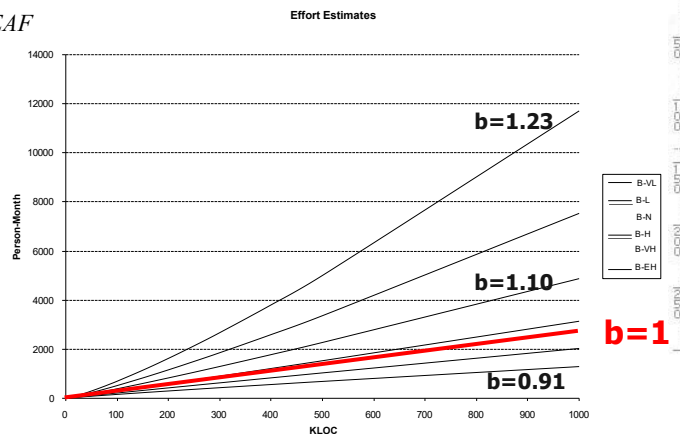
$$0 \leq PREC \leq 6.20$$

$$0 \leq FLEX \leq 5.07$$

$$0 \leq RESL \leq 7.07$$

$$0 \leq TEAM \leq 5.48$$

$$0 \leq PMAT \leq 7.80$$



49

Cost Drivers /1

17 Cost Drivers

- **Product Factors**
 - Required Software Reliability (RELY)
 - Data Base Size (DATA)
 - Product Complexity (CPLX)
 - Required Reusability (RUSE)
 - Documentation (DOCU)
- **Platform Factors**
 - Execution Time Constraint (TIME)
 - Main Storage Constraint (STOR)
 - Platform Volatility (PVOL)

50

Cost Drivers /1

- **Personnel factors**
 - Analyst Capability (ACAP)
 - Application Experience (APEX)
 - Programmer Capability (PCAP)
 - Platform Experience (PLEX)
 - Language and Tool Experience (LTEX)
 - Personnel Continuity (PCON)
- **Project Factors**
 - Software Tools (TOOL)
 - Required Schedule (SCED)
 - Multi-Site Development (SITE)

51

Cost Drivers /2

■ Post-Architecture cost drivers: **Product**

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
RELY	Required software reliability	0.82	0.92	1.00	1.10	1.26	-
DATA	Database size	-	0.90	1.00	1.14	1.28	-
CPLX	Product complexity	0.73	0.87	1.00	1.17	1.34	1.74
RUSE	Required reusability	-	0.95	1.00	1.07	1.15	1.24
DOCU	Documentation	0.81	0.91	1.00	1.11	1.23	-

52

52

Cost Drivers /3

■ Post-Architecture cost drivers: **Platform**

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
TIME	Execution time constraint	-	-	1.00	1.11	1.29	1.63
STOR	Main storage constraint	-	-	1.00	1.05	1.17	1.46
PVOL	Platform volatility	-	0.87	1.00	1.15	1.30	-

53

53

Cost Drivers /4

■ Post-Architecture cost drivers: **Personnel**

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
ACAP	Analyst capability	1.42	1.19	1.00	0.85	0.71	-
AEXP	Applications experience	1.22	1.10	1.00	0.88	0.81	-
PCAP	Programmer capability	1.34	1.15	1.00	0.88	0.76	-
PEXP	Platform experience	1.19	1.09	1.00	0.91	0.85	-
LTEX	Language & Tool experience	1.20	1.09	1.00	0.91	0.84	-
PCON	Personnel continuity	1.29	1.12	1.00	0.90	0.81	-

54

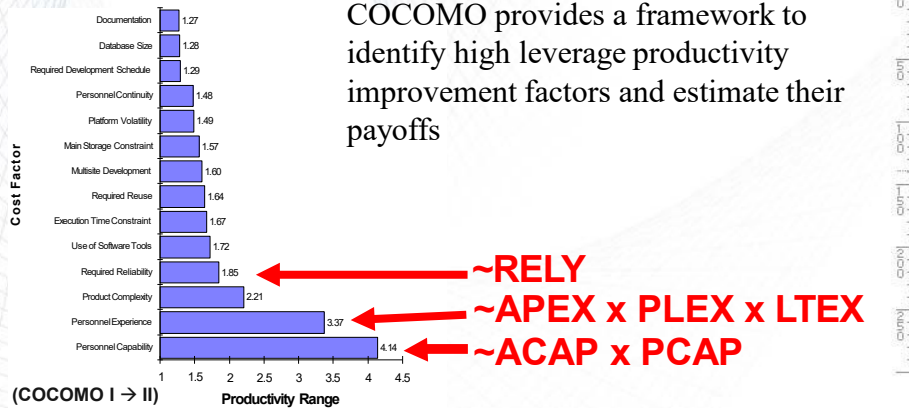
Cost Drivers /5

■ Post-Architecture cost drivers: **Project**

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
TOOL	Software Tools	1.17	1.09	1.00	0.90	0.78	-
SCED	Development Schedule	1.43	1.14	1.00	1.00	1.00	-
SITE	Multi-site development	1.22	1.09	1.00	0.93	0.86	0.80

55

Sensitivity to Cost Drivers



56

COCOMO II

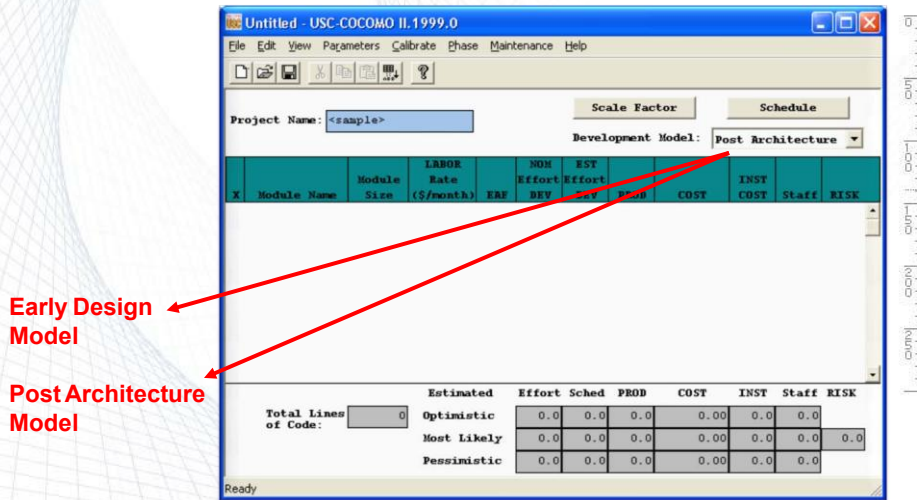
Example & Tool

<http://softwarecost.org/tools/COCOMO/>

<https://strs.grc.nasa.gov/repository/forms/cocomo-calculation/>

57

COCOMO II: The Tool



58

Example: Electronic Bookshop

- Objective: sell books online
- Specifications for the **Application Composition Model** (→ *Object Points*):
 - Number and complexity of screens and reports
 - Number of acquired components
- Specifications for the **Early Design Model** and **Post Architecture Model** (→ *Function Points*):
 - External inputs (EI), External outputs (EO), External inquiries (EQ), External interface files (EIF), Internal logical files (ILF)

59

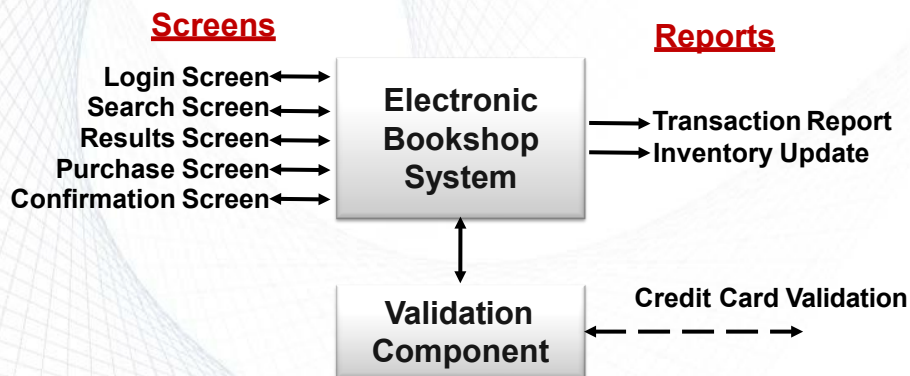
COCOMO II - Example

COCOMO II - Example

Application Composition Model

60

High-Level Specification



61

Application Composition Model

■ Effort

$$E = OP / PROD$$

where

- *OP* is the object point
- *PROD* is the productivity rate
- To determine *OP*, we need to estimate the number of **screens** and **reports** and data inputs for each of them
- Also we need to find the number of acquired components

<i>Developers' experience and capability</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>
<i>PROD</i>	4	7	13	25	50

62

ACM Example: Screens

<i>Screens</i>	<i>Data Inputs</i>
Title Screen	Enter
Search Screen	Title, Author, Publisher, ISBN
Results Screen	Purchase
Purchase Screen	First Name, Last Name, Address, Phone Number, City, Province, Country, Postal Code, E-mail Address, Credit Card Number, Expiration Date
Confirmation Screen	New Search, Print

63

ACM Example: Reports

<i>Reports</i>	<i>Data Entries</i>
Transaction Report	First Name, Last Name, Address, Phone Number, City, Province, Country, Postal Code, E-mail Address, Credit Card Number, Expiration Date, Title, Author, Publisher, ISBN, price
Inventory Update	Title, Author, Publisher, ISBN, price
Credit Card Validation	Credit Card Number, Expiration Date

Assume this is handled by an acquired component

64

Complexity of Screens/Reports

Screen Number of views contained	Number and source of data tables		
	Total <4 <2 servers <2 clients	Total <8 2-3 servers 3-5 clients	Total 8+ >3 servers >5 clients
	Simple	Simple	Medium
< 3	1	1	2
3 - 7	1	2	3
8+	2	3	3

Report Number of sections contained	Number and source of data tables		
	Total <4 <2 servers <2 clients	Total <8 2-3 servers 3-5 clients	Total 8+ >3 servers >5 clients
	Simple	Simple	Medium
0 - 1	2	2	5
2 - 3	2	5	8
4+	5	8	8

- Based on Object Point (OP) complexity level definitions for Screens & Reports:
 - Simple:** Login Screen (1), Search Screen (1), Results Screen (1), Confirmation Screen (1).
 - Medium:** Purchase Screen (2), Transaction Report (5), Inventory Update (5).
- Credit Card Validation is an acquired component (10).

65

Effort Estimation

Object type	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
Component	-	-	10

Simple: 4 screens

Medium: 1 screen, 2 reports

Difficult: 1 COTS (Credit Card Validation)

Object Points (OP):

$$OP = (4 \times 1) + (1 \times 2) + (2 \times 5) + (1 \times 10) = 36$$

Development team consists of:

- 4 entry-level programmers who have 2+ years of experience in C++ and web programming
- However, they have no experience working together
- Thus, we expect team productivity to be low in the beginning, but improve throughout the project (→ nominal)

- For the Application Composition Model, we estimate the average productivity rate PROD to be nominal. Thus the project effort is:

$$\text{Effort} = 36 / 13 = 2.77 \text{ [PM]}$$

66

COCOMO II

Solve the following exercises:

EXERCISE 1

VNA Airline Sales system is to be built in PHP after reusing the 16% of its component. Estimate the efforts using *application composition stage* given the following requirements:

It requires 3 screens and 2 reports. The assessment on the developers shows that the developer's experience is very low and has a productivity rate of 5.5.

Name	Objects	Complexity	Weight
Booking	Screen	Simple	1
Pricing	Screen	Simple <i>Medium</i>	3
Availability	Screen	Medium	2
Sales	2 Reports	Medium	4 <i>5</i>

67

EXERCISE 2

VNA Airline Sales system is to be built in C# (1FP=180 SLOC). Estimate the efforts using *early design* and *post-architecture design* given the following requirements:

Name	External user types	Complexity	FP
Booking	External output type	Low	4
Pricing	External inquiry type	Low	3
Availability	External inquiry type	Medium	4
Sales	External output type	Medium	5

Influence Factors	Rating
F2	1
F3	2
F5	2
F9	4
F10	5
F11	3

Name	Value
Precedentedness	0.01
Flexibility	0.01
Significant risks eliminated	0.03
Team interaction process	0.02
Process maturity	0.04

Identifier (Early design)	Name	Values
RCPX	product Reliability and ComPleXity	0.75
RUSE	required reusability	1.0
PDIF	Platform DIfficulty	1.1
PERS	PERSonnel capability	0.75
PREX	PeRsonnel EXperience	0.65
FCIL	FaCILities available	1.0
SCED	SCHEDule pressure	1.2



68

Group Discussion
Work in group of 4 students
Thảo luận theo nhóm 4 sinh viên
(15 phút)

69

Bài tập về ước tính COCOMO II

Yêu cầu: Dựa trên thông tin dưới đây, hãy sử dụng mô hình COCOMO II để ước tính nỗ lực (effort) và thời gian phát triển sản phẩm phần mềm.

Thông tin dự án:

- Kích thước của phần mềm: 120 KSLOC (Thousand Source Lines of Code).
- Độ phức tạp của phần mềm: Trung bình đến cao.
- Kinh nghiệm của nhóm phát triển :
 - Kinh nghiệm với loại dự án tương tự: Thấp.
 - Kinh nghiệm lập trình: Trung bình.
- Công nghệ và công cụ: Dự án sẽ sử dụng công nghệ hiện đại và các công cụ mới.

Yếu tố môi trường làm việc (EM) được đánh giá như sau (chỉ đưa ra một số ví dụ):

- Thuộc tính về nhân sự:
 - Hệ số hợp tác nhóm (TEAM): 1.0 (Trung bình).
 - Hệ số kinh nghiệm vùng (PERS): 0.9 (Thấp).
- Thuộc tính về sản phẩm:
 - Độ phức tạp của phần mềm (CPLX): 1.0 (Trung bình).

70

Size Estimation: Critics

- Can the existing algorithmic models be applied to a wide range of projects?
- It is suggested that a model is acceptable if 75% of the predicted values fall within 25% of their actual values (Fenton, 1997).
- Unfortunately most of the models are insufficient based on this criteria.



71

71

Conclusions

- Software cost estimation is supposed to be an integral part of the software development process.
- Cost estimation models can be used to represent the relationship between *effort* and a primary cost factor such as *size*.
- Cost drivers are used to adjust the preliminary estimate provided by the primary cost factor.
- Software cost estimation models usually suffer from some common problems:
 - Models are based on empirical results rather than theory.
 - Models are often complex and rely heavily on size estimation.
- **Cost estimation models: You cannot live with them and without them!**

75

References and Assignment

1. Lincoln, C. R. (2005). Best Practices in Software Measurement: How to Use Metrics to Improve Project and Process Performance. *Software Quality Professional*, 7(3), 48. (pp. 95-131, pp.219-300)
2. Watch the below video and answer the following question:
<https://www.youtube.com/watch?v=7nTxd129ePY>

Summarize the contents of the lecture in the video!

76

Bài tập

Size = 200 KLOC

Effort = $a * \text{Size}^b * C$

Cost drivers:

Low reliability .88

High product complexity 1.15

Low application experience 1.13

High programming language experience .95

Organic —

Semidetach

Embedded .